

Django Full Stack Learning Guide

From Zero to Associate Software Engineer
Includes L1 & L2 Interview Preparation

What you will learn:

- Python & Django fundamentals
- Models, Views, Templates (MVT)
- Django ORM & database design
- Forms, Authentication & Security
- Django REST Framework (APIs)
- Frontend integration (HTML/CSS/JS & React)
- Caching, Celery & performance
- Testing & Deployment (Docker, Nginx, Gunicorn)
- 100+ L1 & L2 Interview Questions with answers
- Coding problems & system design basics

Table of Contents

1.	Python Prerequisites for Django	3
2.	Getting Started with Django	5
3.	Django Architecture (MVT)	7
4.	URLs, Views & Templates	9
5.	Models & the ORM	13
6.	Forms & Validation	19
7.	Admin Interface	21
8.	Authentication & Authorization	22
9.	Static Files, Media & Frontend	25
10.	Django REST Framework (DRF)	27
11.	Full Stack Integration (Django + React)	33
12.	Middleware, Signals & Management Commands	35
13.	Caching & Performance Optimization	37
14.	Celery & Background Tasks	39
15.	Testing in Django	40
16.	Security Best Practices	42
17.	Deployment (Docker, Nginx, Gunicorn)	44
18.	Git & CI/CD Basics	47
19.	L1 Interview Questions (Junior)	48
20.	L2 Interview Questions (Associate)	54
21.	Coding Problems & Solutions	60
22.	System Design Basics	63
23.	Mock Interview Scenarios	65
24.	Final Checklist & Resources	67

1. Python Prerequisites for Django

Before diving into Django, you must be comfortable with core Python. Django relies heavily on classes, decorators, and Python idioms. This chapter is a quick refresher of the Python features you will use every day in Django.

1.1 Core Python Concepts

- **Variables & data types:** int, float, str, bool, list, dict, tuple, set.
- **Control flow:** if/elif/else, for, while, comprehensions.
- **Functions:** def, *args, **kwargs, default arguments, lambda.
- **OOP:** classes, inheritance, super(), self, dunder methods (`__init__`, `__str__`).
- **Modules & packages:** import, from ... import, `__init__.py`.
- **Exceptions:** try/except/finally, raise, custom exceptions.
- **File I/O:** open(), context managers (with).

1.2 Object-Oriented Python Example

```
class Animal:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    def speak(self):
        return f"{self.name} says {self.sound}"

class Dog(Animal):
    def __init__(self, name):
        super().__init__(name, sound="Woof")
    def fetch(self):
        return f"{self.name} fetches the ball"

d = Dog("Rex")
print(d.speak())    # Rex says Woof
print(d.fetch())    # Rex fetches the ball
```

1.3 Decorators

Django uses decorators extensively (`@login_required`, `@api_view`, `@property`). A decorator is a function that wraps another function to extend its behaviour without modifying its source.

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log_call
def greet(name):
    return f"Hello {name}"

greet("Alex") # prints "Calling greet" then returns "Hello Alex"
```

1.4 Virtual Environments

Always work inside a virtual environment to isolate dependencies:

```
# Create and activate a venv
python -m venv venv
source venv/bin/activate      # Linux / macOS
venv\Scripts\activate        # Windows

# Install packages
pip install django

# Save dependencies
pip freeze > requirements.txt

# Reproduce on another machine
pip install -r requirements.txt
```

Note: If you skip the venv step, your global Python becomes polluted and different projects may break each other. Always use a venv.

2. Getting Started with Django

2.1 What is Django?

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the "**batteries included**" philosophy: ORM, admin panel, authentication, forms, security, sessions, caching, and more are built in.

Why Django?

- **Fast development:** common features ship out of the box.
- **Secure by default:** protection against XSS, CSRF, SQL injection, clickjacking.
- **Scalable:** used by Instagram, Pinterest, Mozilla, Disqus, Spotify.
- **DRY:** "Don't Repeat Yourself" is baked into the design.
- **Great ORM:** write Python instead of SQL.
- **Massive ecosystem:** DRF, Channels, Celery, etc.

2.2 Installation & First Project

```
# Install Django
pip install django

# Verify
django-admin --version          # e.g. 5.0.x

# Create a project
django-admin startproject mysite
cd mysite

# Run the dev server
python manage.py runserver
# Open http://127.0.0.1:8000
```

2.3 Project Layout

```
mysite/
├── manage.py          # CLI entry point (runserver, migrate, etc.)
├── mysite/
│   ├── __init__.py
│   ├── settings.py    # Configuration (DBs, INSTALLED_APPS, ...)
│   ├── urls.py        # Root URL routing
│   ├── asgi.py        # ASGI entry point (async servers)
│   └── wsgi.py        # WSGI entry point (gunicorn, uwsgi)
```

2.4 Project vs App

A **project** is the whole web application. An **app** is a self-contained module inside that project (e.g. *blog*, *accounts*, *payments*). A project contains many apps; an app can be reused across projects.

```
# Create an app
python manage.py startapp blog

# Register it in settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
```

```
"django.contrib.sessions",
"django.contrib.messages",
"django.contrib.staticfiles",
"blog",          # <-- your app
]
```

2.5 App Layout

```
blog/
■■■ migrations/          # Schema changes
■■■ __init__.py
■■■ admin.py             # Admin registrations
■■■ apps.py              # App config
■■■ models.py            # Database models
■■■ tests.py             # Tests
■■■ views.py             # Request handlers
■■■ urls.py              # (create yourself) URL routing for this app
```

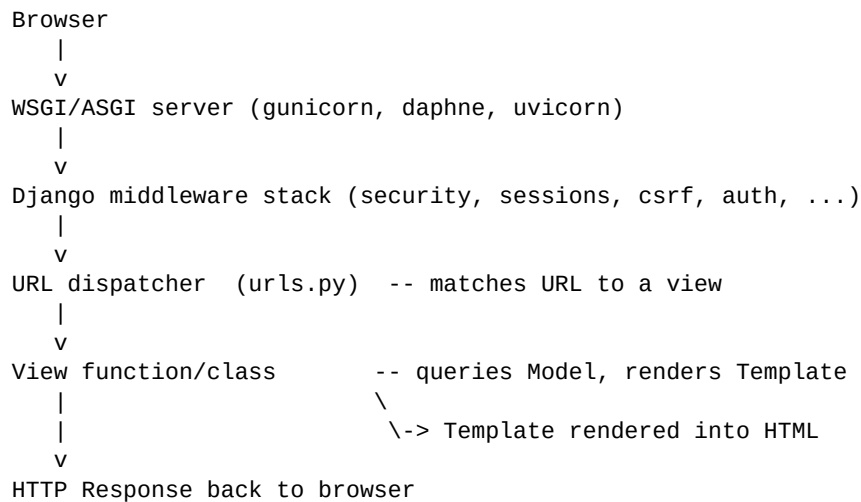
3. Django Architecture (MVT)

Django follows the **Model-View-Template (MVT)** pattern, a Django-specific variation of MVC.

3.1 The Three Layers

- **Model:** defines the data structure; talks to the database via the ORM.
- **View:** Python function/class that handles a request and returns a response. Note: this is NOT the "View" of MVC — it is closer to a "controller".
- **Template:** HTML file with placeholders & logic. Equivalent to MVC's "View".

3.2 Request-Response Cycle



3.3 settings.py Essentials

```
# mysite/settings.py
DEBUG = True                # NEVER True in production
ALLOWED_HOSTS = ["*"]      # restrict in production
SECRET_KEY = "..."        # keep secret; load from env

INSTALLED_APPS = [...]
MIDDLEWARE = [...]
TEMPLATES = [...]
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "mydb", "USER": "postgres",
        "PASSWORD": "secret", "HOST": "localhost", "PORT": "5432",
    }
}
STATIC_URL = "/static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Note: In production, load `SECRET_KEY`, `DEBUG`, and DB credentials from environment variables (use `python-decouple` or `django-environ`). Never commit secrets to Git.

4. URLs, Views & Templates

4.1 URL Routing

URL patterns connect a URL to a view. The project's *urls.py* typically delegates to each app's *urls.py* via *include()*.

```
# mysite/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("blog/", include("blog.urls")),      # app-level
]

# blog/urls.py
from django.urls import path
from . import views

app_name = "blog"
urlpatterns = [
    path("", views.index, name="index"),
    path("<int:pk>/", views.detail, name="detail"),
    path("create/", views.create, name="create"),
    path("<int:pk>/edit/", views.edit, name="edit"),
]
```

Path Converters

- **str** — matches any non-empty string excluding '/' (default).
- **int** — matches positive integers, captured as Python int.
- **slug** — ASCII letters, numbers, hyphens, underscores.
- **uuid** — matches a UUID.
- **path** — like str but allows '/'.

4.2 Function-Based Views (FBV)

```
# blog/views.py
from django.shortcuts import render, get_object_or_404, redirect
from .models import Post
from .forms import PostForm

def index(request):
    posts = Post.objects.all().order_by("-created_at")
    return render(request, "blog/index.html", {"posts": posts})

def detail(request, pk):
    post = get_object_or_404(Post, pk=pk)
    return render(request, "blog/detail.html", {"post": post})

def create(request):
    if request.method == "POST":
        form = PostForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect("blog:index")
    else:
        form = PostForm()
    return render(request, "blog/form.html", {"form": form})
```


4.3 Class-Based Views (CBV)

CBVs encapsulate logic in classes; you inherit and override methods. They are more reusable and shorter for common patterns.

```
from django.views.generic import ListView, DetailView, CreateView, UpdateView
from django.urls import reverse_lazy
from .models import Post

class PostList(ListView):
    model = Post
    template_name = "blog/index.html"
    context_object_name = "posts"
    paginate_by = 10

class PostDetail(DetailView):
    model = Post
    template_name = "blog/detail.html"

class PostCreate(CreateView):
    model = Post
    fields = ["title", "body"]
    template_name = "blog/form.html"
    success_url = reverse_lazy("blog:index")
```

FBV vs CBV — Which to use?

Aspect	FBV	CBV
Readability	Easy for beginners	Slight learning curve
Reusability	Manual	Inheritance / mixins
Boilerplate	More	Less for CRUD
Customisation	Very explicit	Override methods / mixins
Best for	Simple, one-off views	CRUD & repetitive logic

4.4 Templates (DTL)

Django Template Language uses `{{ var }}` for variables, `{% tag %}` for tags, and `|filter` for filters.

```
<!-- blog/templates/blog/base.html -->
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}My Blog{% endblock %}</title>
    {% load static %}
    <link rel="stylesheet" href="{% static 'blog/style.css' %}">
</head>
<body>
    <nav>
        <a href="{% url 'blog:index' %}">Home</a>
        {% if user.is_authenticated %}
            <span>Hi {{ user.username }}</span>
            <a href="{% url 'logout' %}">Logout</a>
        {% else %}
            <a href="{% url 'login' %}">Login</a>
        {% endif %}
    </nav>
    {% block content %}{% endblock %}
</body>
```

```

</html>

<!-- blog/templates/blog/index.html -->
{% extends "blog/base.html" %}
{% block title %}Posts{% endblock %}
{% block content %}
  <h1>All Posts</h1>
  {% for post in posts %}
    <article>
      <h2><a href="{% url 'blog:detail' post.pk %}">{{ post.title }}</a></h2>
      <p>{{ post.body|truncatewords:30 }}</p>
      <small>{{ post.created_at|date:"M d, Y" }}</small>
    </article>
  {% empty %}
    <p>No posts yet.</p>
  {% endfor %}
{% endblock %}

```

Common Template Tags & Filters

- **{% if/elif/else/endif %}, {% for/empty/endfor %}**
- **{% url 'name' arg %}** — reverse URL lookup
- **{% csrf_token %}** — required inside every <form method="post">
- **{% include 'partial.html' %}**
- **{% extends 'base.html' %}** + {% block name %}...{% endblock %}
- **Filters:** length, upper, lower, default, date, truncatewords, linebreaks, safe, escape, join, pluralize

5. Models & the ORM

Models are the single source of truth for your data. Django builds the database schema, the admin, forms, and the ORM from them.

5.1 Defining a Model

```
# blog/models.py
from django.db import models
from django.contrib.auth.models import User

class Category(models.Model):
    name = models.CharField(max_length=80, unique=True)
    slug = models.SlugField(max_length=80, unique=True)

    class Meta:
        verbose_name_plural = "Categories"

    def __str__(self):
        return self.name

class Post(models.Model):
    STATUS = [("draft", "Draft"), ("published", "Published")]

    title = models.CharField(max_length=200)
    slug = models.SlugField(max_length=200, unique=True)
    body = models.TextField()
    author = models.ForeignKey(User, on_delete=models.CASCADE,
                               related_name="posts")
    category = models.ForeignKey(Category, on_delete=models.SET_NULL,
                                 null=True, blank=True)
    tags = models.ManyToManyField("Tag", blank=True)
    status = models.CharField(max_length=10, choices=STATUS,
                              default="draft")
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["status", "-created_at"])]

    def __str__(self):
        return self.title
```

5.2 Common Field Types

Field	Use
CharField(max_length=N)	Short text, requires max_length
TextField	Long, unbounded text
IntegerField, FloatField, DecimalField	Numbers
BooleanField	True/False
DateField, DateTimeField, TimeField	Dates & times
EmailField, URLField, SlugField	Validated text fields
FileField, ImageField	Uploads (need MEDIA_*)
JSONField	Native JSON column (PG, MySQL 5.7+)
ForeignKey, OneToOneField, ManyToManyField	Relationships

5.3 Relationships

- **ForeignKey** — many-to-one (e.g. many Posts have one Author).
- **OneToOneField** — one-to-one (e.g. Profile to User).
- **ManyToManyField** — many-to-many (e.g. Posts to Tags).
- **on_delete** — what happens when the referenced row is deleted: *CASCADE*, *PROTECT*, *SET_NULL*, *SET_DEFAULT*, *DO_NOTHING*.
- **related_name** — name of the reverse accessor: *user.posts.all()*.

5.4 Migrations

```
# After editing models.py:
python manage.py makemigrations      # create a migration file
python manage.py migrate             # apply pending migrations

# Inspect SQL that will run
python manage.py sqlmigrate blog 0001

# Roll back to a previous migration
python manage.py migrate blog 0001

# List migration status
python manage.py showmigrations
```

Note: Always commit your migration files to Git. Never delete a migration that has been applied in production — create a new one.

5.5 QuerySet Basics (CRUD)

```
# CREATE
post = Post.objects.create(title="Hello", body="...", author=user)

# READ
Post.objects.all()
Post.objects.get(pk=1)                # exactly one or DoesNotExist
Post.objects.filter(status="published") # QuerySet
Post.objects.exclude(status="draft")
Post.objects.first(), .last(), .count(), .exists()

# UPDATE
post.title = "New"; post.save()
Post.objects.filter(status="draft").update(status="published")

# DELETE
post.delete()
Post.objects.filter(status="draft").delete()
```

5.6 Field Lookups

```
Post.objects.filter(title__exact="Hello")
Post.objects.filter(title__iexact="hello")
Post.objects.filter(title__contains="Dja")
Post.objects.filter(title__icontains="dja")
Post.objects.filter(title__startswith="D")
Post.objects.filter(views__gt=100, views__lte=1000)
Post.objects.filter(created_at__year=2025, created_at__month=10)
```

```
Post.objects.filter(category__name__in=["Tech", "Python"])
Post.objects.filter(author__isnull=False)
```

5.7 Q Objects & F Expressions

```
from django.db.models import Q, F

# OR / NOT logic
Post.objects.filter(Q(status="published") | Q(author=user))
Post.objects.filter(~Q(status="draft"))

# Compare two columns or update without race conditions
Post.objects.filter(views__gt=F("comments_count"))
Post.objects.filter(pk=1).update(views=F("views") + 1) # atomic
```

5.8 Aggregations & Annotations

```
from django.db.models import Count, Avg, Sum, Min, Max

# Aggregate over the whole queryset
Post.objects.aggregate(total=Count("id"), avg_views=Avg("views"))
# -> {"total": 42, "avg_views": 137.2}

# Annotate each row
User.objects.annotate(post_count=Count("posts"))
# every user now has .post_count

# Group by + filter on aggregate
User.objects.annotate(c=Count("posts")).filter(c__gte=5)
```

5.9 select_related vs prefetch_related

Avoid the N+1 query problem — the single biggest perf issue in Django apps.

```
# BAD: 1 + N queries (one for posts, one for each post.author)
for p in Post.objects.all():
    print(p.author.username)

# GOOD: single JOIN -- for ForeignKey / OneToOne
Post.objects.select_related("author", "category")

# GOOD: separate query then joined in Python -- for ManyToMany / reverse FK
Post.objects.prefetch_related("tags", "comments")
```

Note: `select_related` performs a SQL JOIN (use for FK / OneToOne — single-valued relations). `prefetch_related` does a second query and joins in Python (use for M2M / reverse FK — multi-valued).

5.10 Custom Managers & QuerySets

```
class PublishedManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(status="published")

class Post(models.Model):
    ...
    objects = models.Manager() # default
    published = PublishedManager() # custom

# Usage
Post.published.all()
```

5.11 Transactions

```
from django.db import transaction

@transaction.atomic
def transfer(a, b, amount):
    a.balance -= amount; a.save()
    b.balance += amount; b.save()

# Or as a context manager
with transaction.atomic():
    order.save()
    payment.save()
```

6. Forms & Validation

6.1 Plain Form

```
# forms.py
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)

    def clean_email(self):
        email = self.cleaned_data["email"]
        if email.endswith("@spam.com"):
            raise forms.ValidationError("That domain is blocked.")
        return email
```

6.2 ModelForm

A **ModelForm** builds the form fields automatically from a model — you only declare what's different.

```
from django import forms
from .models import Post

class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["title", "body", "category", "tags"]
        widgets = {
            "body": forms.Textarea(attrs={"rows": 6, "class": "form-control"}),
        }
```

6.3 Using a Form in a View

```
def contact(request):
    if request.method == "POST":
        form = ContactForm(request.POST)
        if form.is_valid():
            data = form.cleaned_data
            # send email, save to DB, etc.
            return redirect("thanks")
    else:
        form = ContactForm()
    return render(request, "contact.html", {"form": form})
```

6.4 Rendering & CSRF

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}           {# or .as_table, .as_ul #}
    {# field-by-field control: #}
    <div class="field">
        {{ form.title.label_tag }}
        {{ form.title }}
        {{ form.title.errors }}
    </div>
    <button type="submit">Send</button>
</form>
```

Note: Every POST form must contain `{% csrf_token %}`. Django will reject the request with a 403 Forbidden otherwise — that protection stops Cross-Site Request Forgery attacks.

7. Admin Interface

Django's admin is a production-ready CRUD interface generated from your models. Register a model and tweak its options:

```
python manage.py createsuperuser      # create admin user

# blog/admin.py
from django.contrib import admin
from .models import Post, Category, Tag

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("title", "author", "status", "created_at")
    list_filter = ("status", "category", "created_at")
    search_fields = ("title", "body")
    prepopulated_fields = {"slug": ("title",)}
    date_hierarchy = "created_at"
    autocomplete_fields = ("author", "category")
    readonly_fields = ("created_at", "updated_at")

admin.site.register(Category)
admin.site.register(Tag)
```

Note: The admin is great for staff/back-office use but is NOT a substitute for a customer-facing UI. Build proper views/templates for end users.

8. Authentication & Authorization

8.1 The User Model

Django ships a User model with username, email, password (hashed), is_staff, is_active, is_superuser, last_login, date_joined.

8.2 Custom User Model (recommended on day 1)

```
# accounts/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):
    bio = models.TextField(blank=True)
    avatar = models.ImageField(upload_to="avatars/", blank=True)

# settings.py
AUTH_USER_MODEL = "accounts.User"
```

Note: Always create a custom User model BEFORE the first migration. Switching later is painful.

8.3 Built-in Auth Views

```
# urls.py
from django.contrib.auth import views as auth_views
from django.urls import path

urlpatterns = [
    path("login/", auth_views.LoginView.as_view(), name="login"),
    path("logout/", auth_views.LogoutView.as_view(), name="logout"),
    path("password_change/",
         auth_views.PasswordChangeView.as_view(), name="password_change"),
    path("password_reset/",
         auth_views.PasswordResetView.as_view(), name="password_reset"),
]
LOGIN_URL = "/login/"
LOGIN_REDIRECT_URL = "/"
LOGOUT_REDIRECT_URL = "/login/"
```

8.4 Restricting Access

```
from django.contrib.auth.decorators import login_required, permission_required
from django.contrib.auth.mixins import LoginRequiredMixin, PermissionRequiredMixin

# FBV
@login_required
def dashboard(request): ...

@permission_required("blog.add_post", raise_exception=True)
def create_post(request): ...

# CBV
class Dashboard(LoginRequiredMixin, TemplateView):
    template_name = "dashboard.html"

class PostCreate(PermissionRequiredMixin, CreateView):
    permission_required = "blog.add_post"
    model = Post
```

8.5 Signup & Login Manually

```
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.forms import UserCreationForm

def signup(request):
    if request.method == "POST":
        form = UserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            return redirect("home")
    else:
        form = UserCreationForm()
    return render(request, "signup.html", {"form": form})

def my_login(request):
    user = authenticate(request,
                        username=request.POST["username"],
                        password=request.POST["password"])
    if user:
        login(request, user)
        return redirect("home")
    return render(request, "login.html", {"error": "Invalid"})
```

8.6 Groups & Permissions

- Every model auto-gets **add_X**, **change_X**, **delete_X**, **view_X** perms.
- Define custom perms in *Meta.permissions*.
- Use **Groups** to bundle perms; assign users to groups.
- Check programmatically: *user.has_perm("blog.delete_post")*.

9. Static Files, Media & Frontend

9.1 Static vs Media

- **Static** — files YOU ship: CSS, JS, images that belong to your code.
- **Media** — files USERS upload: profile pics, attachments.

9.2 Configuration

```
# settings.py
STATIC_URL = "/static/"
STATICFILES_DIRS = [BASE_DIR / "static"]      # dev sources
STATIC_ROOT = BASE_DIR / "staticfiles"       # collectstatic destination

MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"

# In production:
python manage.py collectstatic
```

9.3 Serving Media in Development

```
# urls.py
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [...]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Note: In production, serve static & media through Nginx or a CDN — never through Django. Use *whitenoise* if you must serve statics from Django itself.

9.4 Bootstrap / Tailwind / Vanilla CSS

For an Associate role, you need solid HTML, CSS and a frontend framework. The fastest combos are:

- **Server-rendered:** Django templates + Bootstrap 5 or Tailwind CSS.
- **SPA frontend:** React/Vue talking to a Django REST API.
- **Hybrid:** Django + HTMX + Alpine.js (very productive lately).

10. Django REST Framework (DRF)

DRF is the de-facto library for building REST APIs in Django. Mastery here is essential for any full-stack/backend role.

10.1 Install & Setup

```
pip install djangorestframework
```

```
# settings.py
INSTALLED_APPS += ["rest_framework"]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.SessionAuthentication",
        "rest_framework.authentication.TokenAuthentication",
    ],
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticatedOrReadOnly",
    ],
    "DEFAULT_PAGINATION_CLASS":
        "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 20,
}
```

10.2 Serializers

Serializers convert Django models <-> JSON, and validate input.

```
# serializers.py
from rest_framework import serializers
from .models import Post

class PostSerializer(serializers.ModelSerializer):
    author = serializers.ReadOnlyField(source="author.username")

    class Meta:
        model = Post
        fields = ["id", "title", "body", "author", "status", "created_at"]
        read_only_fields = ["created_at"]

    def validate_title(self, value):
        if len(value) < 5:
            raise serializers.ValidationError("Title too short.")
        return value
```

10.3 The Four View Styles

DRF offers 4 ways to write API views, from most explicit to most concise:

(a) Function-Based @api_view

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status

@api_view(["GET", "POST"])
def post_list(request):
    if request.method == "GET":
        qs = Post.objects.all()
        return Response(PostSerializer(qs, many=True).data)
    s = PostSerializer(data=request.data)
```

```
s.is_valid(raise_exception=True)
s.save(author=request.user)
return Response(s.data, status=status.HTTP_201_CREATED)
```

(b) APIView class

```
from rest_framework.views import APIView

class PostListAPI(APIView):
    def get(self, request):
        qs = Post.objects.all()
        return Response(PostSerializer(qs, many=True).data)

    def post(self, request):
        s = PostSerializer(data=request.data)
        s.is_valid(raise_exception=True)
        s.save(author=request.user)
        return Response(s.data, status=201)
```

(c) Generic Views

```
from rest_framework import generics

class PostList(generics.ListCreateAPIView):
    queryset = Post.objects.all()
    serializer_class = PostSerializer

    def perform_create(self, serializer):
        serializer.save(author=self.request.user)

class PostDetail(generics.RetrieveUpdateDestroyAPIView):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
```

(d) ViewSets + Routers (most concise)

```
from rest_framework import viewsets
from rest_framework.routers import DefaultRouter

class PostViewSet(viewsets.ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer

# urls.py
router = DefaultRouter()
router.register(r"posts", PostViewSet)
urlpatterns = [path("api/", include(router.urls))]
# auto-generates: GET/POST /api/posts/    GET/PUT/PATCH/DELETE /api/posts/<pk>/
```

10.4 Authentication Options

- **SessionAuthentication** — uses Django sessions (great when browser is your client).
- **TokenAuthentication** — simple long-lived token in the Authorization header.
- **JWT (djangorestframework-simplejwt)** — stateless short-lived access + refresh tokens. Standard for SPA/mobile.
- **OAuth2 (django-oauth-toolkit)** — third-party auth flows.

JWT Example

```
pip install djangorestframework-simplejwt

# settings.py
REST_FRAMEWORK["DEFAULT_AUTHENTICATION_CLASSES"] = [
```

```

        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ]

# urls.py
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
urlpatterns += [
    path("api/token/", TokenObtainPairView.as_view()),
    path("api/token/refresh/", TokenRefreshView.as_view()),
]

# Client sends: Authorization: Bearer <access_token>

```

10.5 Permissions

```

from rest_framework import permissions

class IsOwnerOrReadOnly(permissions.BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in permissions.SAFE_METHODS:
            return True
        return obj.author == request.user

class PostViewSet(viewsets.ModelViewSet):
    permission_classes = [permissions.IsAuthenticatedOrReadOnly,
                          IsOwnerOrReadOnly]
    ...

```

10.6 Pagination, Filtering, Searching, Ordering

```

pip install django-filter

REST_FRAMEWORK["DEFAULT_FILTER_BACKENDS"] = [
    "django_filters.rest_framework.DjangoFilterBackend",
    "rest_framework.filters.SearchFilter",
    "rest_framework.filters.OrderingFilter",
]

class PostViewSet(viewsets.ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    filterset_fields = ["status", "category"]
    search_fields = ["title", "body"]
    ordering_fields = ["created_at", "title"]

# GET /api/posts/?status=published&search=django&ordering=-created_at&page=2

```

10.7 Throttling

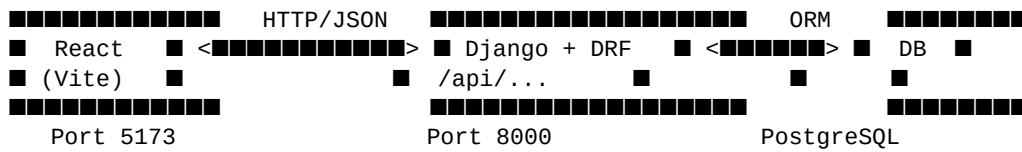
```

REST_FRAMEWORK.update({
    "DEFAULT_THROTTLE_CLASSES": [
        "rest_framework.throttling.AnonRateThrottle",
        "rest_framework.throttling.UserRateThrottle",
    ],
    "DEFAULT_THROTTLE_RATES": {"anon": "60/min", "user": "1000/day"},
})

```

11. Full Stack Integration (Django + React)

11.1 Architecture



11.2 CORS Setup

```
pip install django-cors-headers
```

```
# settings.py
INSTALLED_APPS += ["corsheaders"]
MIDDLEWARE.insert(0, "corsheaders.middleware.CorsMiddleware")

CORS_ALLOWED_ORIGINS = ["http://localhost:5173",
                        "https://yourapp.com"]
CORS_ALLOW_CREDENTIALS = True
```

11.3 React Side (fetch with JWT)

```
// api.js
const API = "http://localhost:8000/api";

export async function login(username, password) {
  const r = await fetch(`${API}/token/`, {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({username, password}),
  });
  const data = await r.json();
  localStorage.setItem("access", data.access);
  localStorage.setItem("refresh", data.refresh);
}

export async function getPosts() {
  const r = await fetch(`${API}/posts/`, {
    headers: {Authorization: `Bearer ${localStorage.getItem("access")}`},
  });
  return r.json();
}
```

11.4 Common Patterns

- **Optimistic UI** — update UI before the API returns; rollback on error.
- **React Query / SWR** — caches API responses and revalidates.
- **Refresh tokens** — silently get a new access token before it expires.
- **Protected routes** — redirect to login if no token / 401 received.

12. Middleware, Signals & Management Commands

12.1 Middleware

Middleware sits between the WSGI server and the view. Each request passes through every middleware on the way in and out.

```
# myapp/middleware.py
import time

class TimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.time()
        response = self.get_response(request)
        response["X-Response-Time-ms"] = int((time.time() - start) * 1000)
        return response

# settings.py
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
    "myapp.middleware.TimingMiddleware",
]
```

12.2 Signals

Signals let decoupled apps react to events: a model was saved, a user logged in, a migration ran, etc.

```
# accounts/signals.py
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth import get_user_model
from .models import Profile

User = get_user_model()

@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)

# accounts/apps.py
from django.apps import AppConfig

class AccountsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "accounts"
    def ready(self):
        from . import signals
```

Note: Use signals sparingly. They make code hard to trace. Prefer explicit function calls when possible. Good signal uses: cross-app coupling, third-party integrations.

12.3 Custom Management Commands

```
# myapp/management/commands/cleanup_drafts.py
from django.core.management.base import BaseCommand
from myapp.models import Post

class Command(BaseCommand):
    help = "Delete draft posts older than 30 days"

    def add_arguments(self, parser):
        parser.add_argument("--days", type=int, default=30)

    def handle(self, *args, **opts):
        from django.utils import timezone
        from datetime import timedelta
        cutoff = timezone.now() - timedelta(days=opts["days"])
        deleted, _ = Post.objects.filter(
            status="draft", created_at__lt=cutoff).delete()
        self.stdout.write(self.style.SUCCESS(f"Deleted {deleted} drafts"))

# Run:
python manage.py cleanup_drafts --days=60
```

13. Caching & Performance Optimization

13.1 Cache Backends

```
# settings.py
CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.redis.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
    }
}
```

13.2 Three Levels of Cache

```
# 1. Low-level cache API
from django.core.cache import cache
cache.set("key", value, timeout=300)
value = cache.get("key")

# 2. Per-view cache
from django.views.decorators.cache import cache_page

@cache_page(60 * 15)  # 15 min
def index(request): ...

# 3. Template fragment cache
{% load cache %}
{% cache 500 sidebar %}
... expensive template ...
{% endcache %}
```

13.3 Database Optimisation Checklist

- Use **select_related** & **prefetch_related** to kill N+1 queries.
- Add **db_index=True** or **Meta.indexes** on columns used in WHERE/ORDER BY.
- Use **only()** / **defer()** to fetch fewer columns.
- Use **values()** / **values_list()** to skip building model instances.
- Use **bulk_create** / **bulk_update** for many rows.
- Use **F()** expressions for atomic updates without races.
- Inspect SQL with **django-debug-toolbar** in dev.
- Use **EXPLAIN ANALYZE** on slow queries.
- Cache expensive reads in Redis.

13.4 Production Server Optimisations

- Run Gunicorn with multiple workers: *workers = 2 * CPUs + 1*.
- Serve static via Nginx or CDN, never Django.
- Use connection pooling (pgbouncer) for Postgres.
- Enable Gzip in Nginx.
- Use `DATABASES['default']['CONN_MAX_AGE']` for persistent DB connections.

14. Celery & Background Tasks

Use Celery for long-running tasks (emails, image processing, ML inference, reports). Celery offloads work to background workers via a message broker (Redis or RabbitMQ).

14.1 Setup

```
pip install celery redis

# mysite/celery.py
import os
from celery import Celery

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "mysite.settings")
app = Celery("mysite")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()

# mysite/__init__.py
from .celery import app as celery_app
__all__ = ("celery_app",)

# settings.py
CELERY_BROKER_URL = "redis://localhost:6379/0"
CELERY_RESULT_BACKEND = "redis://localhost:6379/0"
```

14.2 Write & Call a Task

```
# blog/tasks.py
from celery import shared_task
from django.core.mail import send_mail

@shared_task
def send_welcome_email(user_id):
    from django.contrib.auth import get_user_model
    u = get_user_model().objects.get(pk=user_id)
    send_mail("Welcome!", "Hi " + u.username,
            "noreply@x.com", [u.email])

# views.py – fire-and-forget
send_welcome_email.delay(user.id)

# Start a worker
celery -A mysite worker -l info

# Periodic tasks (Celery Beat)
celery -A mysite beat -l info
```

15. Testing in Django

15.1 TestCase Basics

```
# blog/tests.py
from django.test import TestCase, Client
from django.urls import reverse
from django.contrib.auth import get_user_model
from .models import Post

User = get_user_model()

class PostModelTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.user = User.objects.create_user("alice", password="x")

    def test_str(self):
        p = Post.objects.create(title="Hello", body="...", author=self.user)
        self.assertEqual(str(p), "Hello")

class PostViewTests(TestCase):
    def setUp(self):
        self.client = Client()
        self.user = User.objects.create_user("bob", password="x")

    def test_index_returns_200(self):
        resp = self.client.get(reverse("blog:index"))
        self.assertEqual(resp.status_code, 200)

    def test_create_requires_login(self):
        resp = self.client.get(reverse("blog:create"))
        self.assertRedirects(resp, "/login/?next=/blog/create/")
```

15.2 DRF API Tests

```
from rest_framework.test import APITestCase
from rest_framework import status

class PostAPITests(APITestCase):
    def setUp(self):
        self.user = User.objects.create_user("a", password="x")
        self.client.force_authenticate(self.user)

    def test_create(self):
        r = self.client.post("/api/posts/",
                             {"title": "Hello", "body": "B"}, format="json")
        self.assertEqual(r.status_code, status.HTTP_201_CREATED)
        self.assertEqual(Post.objects.count(), 1)
```

15.3 Running Tests

```
python manage.py test          # whole project
python manage.py test blog     # one app
python manage.py test blog.tests.PostModelTests.test_str
python manage.py test --parallel --keepdb  # faster reruns

# Coverage
pip install coverage
coverage run --source=. manage.py test
coverage report -m
coverage html
```

15.4 Pytest Alternative

```
pip install pytest pytest-django
```

```
# pytest.ini
[pytest]
DJANGO_SETTINGS_MODULE = mysite.settings
python_files = tests.py test_*.py *_tests.py

# example
import pytest
@pytest.mark.django_db
def test_post_creation():
    u = User.objects.create_user("z", password="x")
    p = Post.objects.create(title="T", author=u)
    assert p.pk is not None
```

16. Security Best Practices

Django is secure-by-default but only if you don't fight it. Mistakes in this area appear in nearly every interview.

16.1 Built-in Protections

Attack	Django Defence
SQL Injection	ORM parameterises every query
XSS	Templates auto-escape; only <code> safe</code> / <code>mark_safe</code> opt-out
CSRF	<code>CsrfViewMiddleware</code> + <code>{% csrf_token %}</code> on every POST
Clickjacking	<code>XFrameOptionsMiddleware</code> → <code>X-Frame-Options: DENY</code>
Session hijack	Secure, <code>HttpOnly</code> cookies; rotate <code>session_key</code> on login
Host header	<code>ALLOWED_HOSTS</code> check
Password attacks	PBKDF2 hashing + validators + login throttling

16.2 Production Security Checklist

```
# settings.py (prod)
DEBUG = False
ALLOWED_HOSTS = ["yourdomain.com"]
SECRET_KEY = os.environ["SECRET_KEY"]

# HTTPS
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True

# Headers
SECURE_CONTENT_TYPE_NOSNIFF = True
SECURE_BROWSER_XSS_FILTER = True
X_FRAME_OPTIONS = "DENY"

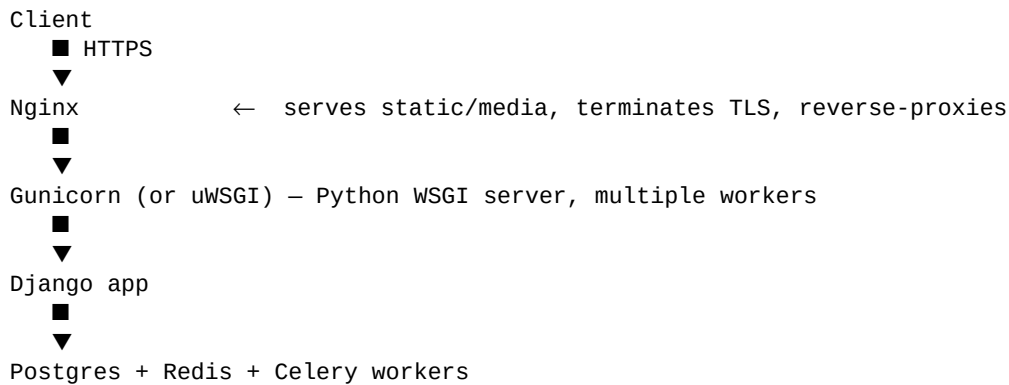
# Run audit
python manage.py check --deploy
```

16.3 Common Vulnerabilities You Must Know

- **Never** disable CSRF with `@csrf_exempt` unless you have token auth.
- **Never** turn off auto-escape; if you must, sanitise input with *bleach*.
- **Never** build SQL with f-strings; use ORM or parameterised *RawSQL*.
- **Never** store passwords in plaintext or use MD5/SHA1.
- **Validate uploads**: check size, `content_type`, scan if needed.
- **Don't** trust user-supplied redirects; validate next URL is internal.

17. Deployment (Docker, Nginx, Gunicorn)

17.1 Production Stack



17.2 Gunicorn

```
pip install gunicorn

gunicorn mysite.wsgi:application \
    --bind 0.0.0.0:8000 \
    --workers 3 \
    --threads 2 \
    --timeout 60 \
    --access-logfile - --error-logfile -
```

17.3 Nginx Config

```
server {
    listen 80;
    server_name yourdomain.com;

    location /static/ { alias /app/staticfiles/; }
    location /media/ { alias /app/media/; }

    location / {
        proxy_pass http://web:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

17.4 Dockerfile

```
FROM python:3.12-slim

ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN python manage.py collectstatic --noinput
```



```
EXPOSE 8000
CMD ["gunicorn", "mysite.wsgi:application",
    "--bind", "0.0.0.0:8000", "--workers", "3"]
```

17.5 docker-compose.yml

```
version: "3.9"
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret
    volumes: ["pg:/var/lib/postgresql/data"]

  redis:
    image: redis:7

  web:
    build: .
    environment:
      - DATABASE_URL=postgres://postgres:secret@db:5432/mydb
      - REDIS_URL=redis://redis:6379/0
    depends_on: [db, redis]
    ports: ["8000:8000"]

  worker:
    build: .
    command: celery -A mysite worker -l info
    depends_on: [db, redis]

  nginx:
    image: nginx:alpine
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
      - ./staticfiles:/app/staticfiles
    ports: ["80:80"]
    depends_on: [web]

volumes:
  pg:
```

17.6 12-Factor / Environment Variables

```
pip install django-environ
```

```
# settings.py
import environ
env = environ.Env()
environ.Env.read_env(BASE_DIR / ".env")

SECRET_KEY = env("SECRET_KEY")
DEBUG      = env.bool("DEBUG", default=False)
DATABASES  = {"default": env.db()}
CACHES     = {"default": env.cache("REDIS_URL")}
```

18. Git & CI/CD Basics

18.1 Essential Git

```
git init
git add . && git commit -m "Initial commit"
git checkout -b feature/login
# ...code...
git add -p && git commit -m "Add login view"
git push origin feature/login
# open Pull Request → review → merge
```

18.2 .gitignore

```
*.pyc
__pycache__/
venv/
.env
*.sqlite3
/staticfiles/
/media/
.DS_Store
.idea/ .vscode/
```

18.3 GitHub Actions Sample

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: testdb
        ports: ["5432:5432"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: python manage.py test
      env:
        DATABASE_URL: postgres://postgres:postgres@localhost:5432/testdb
        SECRET_KEY: ci-secret
```

19. L1 Interview Questions (Junior)

L1 (entry / junior) interviews focus on fundamentals: definitions, syntax, basic flows. You should be able to answer each in 30–60 seconds.

Q. What is Django?

A. Django is a high-level, open-source Python web framework that follows the Model-View-Template (MVT) pattern. It includes an ORM, admin site, authentication, forms, security, and templating out of the box, enabling rapid and secure development.

Q. What is the MVT pattern in Django?

A. Model = data layer; View = Python code that handles requests, queries models, and returns responses; Template = the HTML presentation. Django calls its 'controller' the View and its 'view' the Template, which is why it's MVT instead of MVC.

Q. Difference between a Django project and a Django app?

A. A project is the whole site / configuration. An app is a self-contained module within a project responsible for one feature (e.g. blog, accounts). One project contains many apps; one app can be reused in multiple projects.

Q. What is manage.py?

A. A thin wrapper around django-admin that sets DJANGO_SETTINGS_MODULE for your project. You run admin commands through it: runserver, migrate, makemigrations, createsuperuser, etc.

Q. What is the role of settings.py?

A. It is the central configuration file: INSTALLED_APPS, MIDDLEWARE, DATABASES, TEMPLATES, STATIC_URL, AUTH_USER_MODEL, SECRET_KEY, DEBUG, ALLOWED_HOSTS, etc.

Q. What is the ORM?

A. Object-Relational Mapper — it lets you query and manipulate the database using Python objects instead of writing raw SQL. `Post.objects.filter(status='published')` is translated by Django into a parameterised SQL SELECT.

Q. What is a QuerySet?

A. A lazy, chainable collection of database queries. It is not evaluated until you iterate over it, slice it, or call methods like `len()`, `list()`, or `bool()`. Lazy evaluation lets you compose filters efficiently.

Q. What does on_delete do?

A. It declares what should happen when the referenced row is deleted. Options: CASCADE (delete this row too), PROTECT (raise an error), SET_NULL (set FK to null; field must be nullable), SET_DEFAULT, DO_NOTHING.

Q. Difference between null=True and blank=True?

A. `null=True` allows NULL in the database. `blank=True` allows empty values in forms / validation. For CharField/TextField, use `blank=True` and avoid `null=True` so Django can store "" rather than two empty representations.

Q. What is a migration?

A. A Python file describing a schema change. makemigrations creates the file from your model changes; migrate applies pending migrations to the database.

Q. How do you connect MySQL/PostgreSQL?

A. Install the driver (psycopg2-binary for PG, mysqlclient for MySQL), set DATABASES['default']['ENGINE'] to the appropriate backend, and provide NAME, USER, PASSWORD, HOST, PORT.

Q. What is CSRF and how does Django protect against it?

A. Cross-Site Request Forgery: a malicious site tricks a logged-in user into submitting a request to your site. Django adds a per-session random token via CsrfViewMiddleware; templates inject it with {% csrf_token %}, and POST requests without it are rejected.

Q. What is the difference between GET and POST in Django?

A. GET is for retrieving data (idempotent, safe, visible in URL). POST is for creating/updating data (not idempotent, body data, requires CSRF token in Django).

Q. What is request.POST vs request.GET?

A. Dict-like QueryDict objects that contain form-encoded data submitted via POST or query-string parameters submitted via GET, respectively.

Q. What is the difference between render and redirect?

A. render(request, template, context) returns an HttpResponse with the rendered template. redirect(to) returns an HttpResponseRedirect (302) telling the browser to go to a new URL.

Q. What is a Django template?

A. A text file (usually HTML) with placeholders {{ variable }} and tags {% tag %}. The template engine renders it with a context dictionary into the final HTML.

Q. How do you reverse a URL by name?

A. Use {% url 'app_name:url_name' arg %} in templates or reverse('app_name:url_name', args=[...]) in Python.

Q. What is the difference between {% include %} and {% extends %}?

A. {% include %} inserts another template inline. {% extends %} declares that this template inherits a parent and overrides its {% block %}s.

Q. What is a slug?

A. A short URL-friendly string of letters, numbers, hyphens and underscores — e.g. 'my-first-post'. SlugField validates it.

Q. What does makemigrations vs migrate do?

A. makemigrations creates new migration files from model changes; migrate applies any unapplied migrations to the database.

Q. How do you create a superuser?

A. python manage.py createsuperuser — prompts for username, email, and password.

Q. What is INSTALLED_APPS?

A. A list of all Django apps active in the project. Django uses it to discover models, templates, static files, signals, and admin registrations.

Q. What are signals?

A. Hooks that let decoupled apps react to events. Common signals: `pre_save`, `post_save`, `pre_delete`, `post_delete`, `m2m_changed`, `user_logged_in`.

Q. What is middleware?

A. Layers of code that process every request and response. They sit between the WSGI server and the view, used for auth, sessions, CSRF, GZip, logging, etc.

Q. Difference between authentication and authorization?

A. Authentication = verifying who you are (login). Authorization = verifying what you can do (permissions / groups).

Q. What is the default Django authentication backend?

A. `django.contrib.auth.backends.ModelBackend` — checks username + password against the User model.

Q. How do you protect a view to logged-in users only?

A. Decorate FBVs with `@login_required`, or for CBVs inherit from `LoginRequiredMixin`. Configure `LOGIN_URL` in settings.

Q. What is the difference between SessionAuthentication and TokenAuthentication in DRF?

A. Session auth uses Django's session cookie (great for browser apps on the same domain). Token auth sends an 'Authorization: Token ' header — works well for mobile and SPA clients.

Q. What is a serializer in DRF?

A. An object that converts between complex types (model instances, querysets) and Python primitives that can be rendered to JSON, plus validates incoming data.

Q. What is the difference between APIView and ViewSet?

A. `APIView` is a class with explicit `get/post/put` methods bound to one URL. `ViewSet` bundles `list/create/retrieve/update/destroy` actions and is routed automatically by a Router.

Q. What does DEBUG=True do?

A. Enables detailed error pages with stack traces, disables some security checks, and serves static files via Django's dev server. It must be `False` in production.

Q. What is ALLOWED_HOSTS?

A. A list of host/domain names this Django site can serve. Protects against HTTP Host header attacks. Required when `DEBUG=False`.

Q. What is the use of __str__ in a model?

A. Returns a human-readable representation of the instance. Shown in the admin, in shell output, and anywhere Django needs to render the object as text.

Q. How do you fetch a single object or 404?

A. `from django.shortcuts import get_object_or_404; get_object_or_404(Model, pk=pk)` — raises `Http404` instead of `DoesNotExist`.

Q. Difference between .filter() and .get()?

A. `.filter()` returns a (possibly empty) `QuerySet`. `.get()` returns exactly one object or raises `DoesNotExist` (zero matches) or `MultipleObjectsReturned` (more than one).

Q. How do you serve uploaded files in development?

A. Configure `MEDIA_URL` and `MEDIA_ROOT`, then in `urls.py`: `urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)` inside an `'if settings.DEBUG:'` block.

Q. What is the use of `related_name`?

A. Sets the name of the reverse accessor on a `ForeignKey/ManyToMany`. With `author = ForeignKey(User, related_name='posts')`, you can call `user.posts.all()` instead of `user.post_set.all()`.

Q. What is a context processor?

A. A function that returns a dictionary added to every template's context. Used to expose global data like `'request'`, `'user'`, `'site_settings'` to all templates.

Q. How do you run only one test?

A. `python manage.py test app.tests.MyTestCase.test_method`

Q. What is the difference between class-based and function-based views?

A. FBVs are simple functions taking a request and returning a response — explicit and easy to read. CBVs are classes that group request handlers (`get`, `post`, ...) and offer mixins/inheritance for reuse, reducing boilerplate for CRUD.

20. L2 Interview Questions (Associate)

L2 / Associate Software Engineer interviews go beyond syntax — they probe trade-offs, performance, security and real-world design. Speak in terms of scenarios and trade-offs, not just definitions.

Q. Explain the request-response cycle in Django end-to-end.

A. 1) Browser sends HTTP request to WSGI/ASGI server (Gunicorn/Daphne). 2) Server hands it to Django's WSGI handler. 3) Each middleware processes the request (security, sessions, CSRF, auth). 4) The URL resolver matches a URL pattern and dispatches to a view. 5) The view queries models, runs business logic, renders a template or serialises data. 6) The response travels back through every middleware in reverse. 7) The server writes the HTTP response back to the browser.

Q. How does Django's ORM avoid SQL injection?

A. All ORM queries are parameterised — user data is passed as parameters to the DB driver, never concatenated into the SQL string. Even `.raw()` and `.extra()` accept parameters. The only unsafe escape hatches are string-formatting your own SQL or using `cursor.execute()` with f-strings, which you must avoid.

Q. What is the N+1 query problem and how do you fix it?

A. When you fetch N rows and then access a related object on each, Django issues one extra query per row (N+1 total). Fix with `select_related()` for a SQL JOIN on single-valued relationships, or `prefetch_related()` for multi-valued — which runs a second query and joins in Python.

Q. When do you use `select_related` vs `prefetch_related`?

A. `select_related` does a SQL JOIN and fetches everything in one query; use it for ForeignKey and OneToOneField. `prefetch_related` runs a separate query and joins data in Python; use it for ManyToMany and reverse ForeignKey lookups (where a JOIN would duplicate rows).

Q. Explain Django middleware execution order.

A. On the request side, middleware runs top-to-bottom as listed in `MIDDLEWARE`. On the response side, it runs bottom-to-top — like an onion. Security must be early; middleware that modifies the response (GZip, common) can be later. Custom middleware should be added in a spot that respects these dependencies.

Q. How do Django signals work and when should you avoid them?

A. Signals are an observer pattern: senders dispatch, receivers handle. Useful for decoupling (e.g. creating a profile when a user is saved). Avoid when the logic could be inlined — they make the call graph hidden, are harder to test, and execute synchronously inside the same transaction. Prefer an explicit service function for in-app logic.

Q. What's the difference between `save()` and `update()`?

A. `.save()` loads the row into memory, modifies the Python object, and writes every field back — triggers `pre_save/post_save` signals. `QuerySet.update()` emits a single UPDATE statement at the DB layer for all matching rows, skipping signals and `save()` overrides. Much faster for bulk changes but bypasses model-level logic.

Q. Explain Django's transaction handling.

A. By default, each HTTP request is auto-committed per query. Wrap a block with `@transaction.atomic` to make it a single transaction — either all DB changes succeed or none.

ATOMIC_REQUESTS=True wraps every view in one transaction. Use savepoints (transaction.atomic() nested) for partial rollbacks. Be careful in Celery tasks — they run outside the request transaction.

Q. Difference between OneToOneField, ForeignKey, and ManyToManyField — with examples.

A. OneToOne — exactly one row on each side: User ↔ Profile. ForeignKey — many to one: many Posts to one Author. ManyToMany — many to many: Posts ↔ Tags, implemented as a join table (Django auto-creates one or you can specify a 'through' model for extra fields).

Q. How does Django handle migrations conflicts?

A. When two branches each add a migration with the same parent, you get two leaf migrations. Run python manage.py makemigrations --merge to create a merge migration that depends on both. Always review the merge file before committing.

Q. How would you handle database connection pooling?

A. Django opens a new connection per request by default. Set CONN_MAX_AGE in the DATABASES setting to reuse connections. For high load, place pgbouncer or PgCat between Django and Postgres for true pooling; ensure connection-state issues (like SET commands) don't leak across pool clients.

Q. Explain how Django's authentication system works internally.

A. AuthenticationMiddleware reads the session, looks up the user_id, fetches the User from the DB, and attaches it to request.user. The authenticate() function tries each backend in AUTHENTICATION_BACKENDS; the first to return a User wins. login(request, user) sets request.session['_auth_user_id'] and request.session['_auth_user_backend']. Passwords are hashed with PBKDF2 by default.

Q. How do you implement a custom user model — and why?

A. Subclass AbstractUser (keep username/email/etc.) or AbstractBaseUser (start from scratch); set AUTH_USER_MODEL = 'app.User' in settings before the first migration. You do this so you can add fields (avatar, bio, role) or change the login identifier to email without monkey-patching. Switching afterwards is painful — do it on day one.

Q. What is the difference between JWT and session-based authentication?

A. Sessions: server stores state (session_id ↔ user) in DB/cache; client holds a cookie. Easy to revoke, but requires a session store and couples scaling. JWT: stateless, signed token containing the claims; server validates the signature on every request — no session store, but revocation is harder (you need a blacklist or short-lived tokens + refresh tokens). Sessions for traditional web apps; JWT for SPA/mobile/microservices.

Q. How do you optimise a slow Django view?

A. 1) Profile with django-debug-toolbar or silk to find the hotspot. 2) Eliminate N+1 with select_related/prefetch_related. 3) Index columns used in WHERE / ORDER BY. 4) Use only()/defer()/values() to fetch fewer columns. 5) Cache expensive computations or queries in Redis. 6) Move slow work (emails, reports) to Celery. 7) Paginate large querysets. 8) Use bulk_create/bulk_update for batched writes.

Q. How does caching work in Django and where would you use it?

A. Django offers a low-level cache API, per-view cache, and template fragment cache, with backends like LocMem, Redis, Memcached, DB or file. Use per-view for anonymous heavy pages,

fragment cache for expensive sidebars/widgets, and low-level cache for query results, API payloads, computed values. Always set a TTL and a cache invalidation strategy (signals or explicit `cache.delete` on writes).

Q. What is a Manager and a custom QuerySet — what's the difference?

A. A Manager is the gateway from a model to the DB (`Model.objects`). A QuerySet is the iterable representing a query. You can define custom Manager methods (`Post.objects.published()`) or, more idiomatically, subclass QuerySet and call `as_manager()` so the methods chain (`Post.objects.published().recent()`).

Q. Explain F and Q expressions with use cases.

A. `F()` refers to a database column directly, enabling atomic updates (`Post.objects.filter(pk=1).update(views=F('views')+1)`) without race conditions and column-to-column comparisons. `Q()` builds complex logical conditions with `|`, `&`, `~` — necessary for OR/NOT queries the default filter() can't express.

Q. What happens under the hood when you call `queryset.update()`?

A. Django generates a single `UPDATE ... WHERE ...` statement covering all matched rows; it doesn't load instances into memory, doesn't fire `pre_save/post_save` signals, doesn't run `model.save()` overrides. It returns the number of rows affected. Use when bulk updating with simple value changes.

Q. How do you add an index to a Django model?

A. Either `db_index=True` on a single field, or define `Meta.indexes` for single and composite indexes: `indexes = [models.Index(fields=['status', '-created_at'])]`. Use `UniqueConstraint` for unique composites. Then `makemigrations + migrate` — Django emits `CREATE INDEX`.

Q. Explain Django's deployment architecture.

A. Behind the scenes: Nginx terminates TLS, serves static and media directly, and reverse-proxies dynamic requests to Gunicorn (a WSGI server with multiple worker processes). Gunicorn invokes the Django WSGI app. Django talks to Postgres (often via `pgbouncer`) and Redis. Long-running work goes to Celery workers reading from a Redis/RabbitMQ broker. Logs are shipped to a central system (ELK, CloudWatch). Everything runs in Docker, orchestrated by ECS / Kubernetes / Compose.

Q. How do you secure a Django app for production?

A. `DEBUG=False`; restrict `ALLOWED_HOSTS`; load `SECRET_KEY` from env; enforce HTTPS (`SECURE_SSL_REDIRECT`, `HSTS`); secure cookies (`SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `HttpOnly`, `SameSite`); `X-Frame-Options DENY`; content security policy via `django-csp`; rate-limit with DRF throttling or `django-ratelimit`; validate uploads; sanitize HTML with `bleach`; rotate `SECRET_KEY`/credentials regularly; run `'python manage.py check --deploy'`.

Q. How does DRF's permission system work?

A. On every request, DRF iterates `permission_classes` and calls `.has_permission(request, view)`; on object-level actions, also `.has_object_permission(request, view, obj)`. If any returns `False`, DRF raises 403 (or 401 if unauthenticated). Combine global defaults (`IsAuthenticated`) with custom per-view logic, e.g. `owner-only` edits.

Q. What's the difference between `perform_create` and `create` in DRF?

A. `create()` is the full create flow on a `ViewSet/Generic view` — handles validation, response, status. `perform_create(serializer)` is the hook inside that flow where you actually call `serializer.save()` and can inject extra data (e.g. `author=self.request.user`). Override `perform_create` when you only need to tweak `save()` arguments.

Q. Explain Django Channels and when to use it.

A. Channels extends Django to support ASGI (async) and protocols beyond HTTP — WebSockets, MQTT, Server-Sent Events. Use it for real-time features: chat, live notifications, collaborative editing, live dashboards. It runs on Daphne/Uvicorn; consumers replace views for WebSocket connections; a channel layer (usually Redis) lets workers broadcast messages.

Q. How does Celery integrate with Django and what problems does it solve?

A. Celery is a distributed task queue. Django app fires `task.delay(args)`, which serialises a message to a broker (Redis/RabbitMQ). Worker processes pick up messages and execute tasks. Solves long-running work (emails, PDF/report generation, image processing, ML inference) that would otherwise block the request, plus scheduled jobs via Celery Beat.

Q. How do you handle file uploads at scale?

A. Use `ImageField/FileField` but store on S3 (`django-storages`) instead of local disk; validate size and MIME type; for large files, use presigned URLs so the browser uploads directly to S3 — Django only receives a metadata POST. Run virus scanning out-of-band if needed.

Q. Explain how you'd design pagination for a large API.

A. `PageNumberPagination` is simple but offsets are slow on huge tables. `CursorPagination` is $O(1)$ and consistent under writes — best for feeds. `LimitOffsetPagination` is flexible but suffers the same offset issues. Always set a max page size to avoid abuse.

Q. How do you test a view that requires authentication?

A. In Django tests: `self.client.force_login(user)` or `self.client.login(username=..., password=...)`. In DRF tests: `self.client.force_authenticate(user)` on `APITestCase`. Then call the URL with `self.client.get/post` and assert status / body.

Q. How does Django handle static files in production?

A. Run `python manage.py collectstatic` to copy every app's static/ into `STATIC_ROOT`, then have Nginx serve that path directly — Django doesn't serve static in prod. For containers without Nginx, use `Whitenoise` middleware to serve compressed, hash-named files.

Q. How do you do database-agnostic migrations?

A. Stay inside the ORM and avoid backend-specific column types. For data migrations, write `RunPython(forward, reverse)` using the historical model (`apps.get_model()`). Avoid running raw SQL via `RunSQL` unless it's portable or you guard with router checks.

Q. Explain Django REST Framework's serializer validation pipeline.

A. 1) `is_valid()` runs field-level `to_internal_value` (type/format). 2) field-level `validate_` methods. 3) any `validators=` callables. 4) object-level `validate(attrs)` for cross-field rules. 5) On `save()`, `create()` or `update()` is called. Errors collect into `serializer.errors` as a dict and trigger 400.

Q. What is the difference between AbstractUser and AbstractBaseUser?

A. `AbstractUser` provides the full default fields (`username`, `email`, `password`, `first_name`, `last_name`, `is_staff`, `is_active`, ...) — extend it to add fields. `AbstractBaseUser` is the bare minimum

(password, last_login) — use it when you want to redefine the identity field (e.g. email-only login) and the auth flow.

Q. How do you handle environment-specific settings?

A. Split settings.py into base.py + dev.py + prod.py + test.py, set DJANGO_SETTINGS_MODULE per environment. Or keep one settings.py and read everything from env vars with django-environ / python-decouple. Either way: secrets must come from env, not Git.

Q. Explain the difference between auto_now and auto_now_add.

A. auto_now_add=True sets the field once on creation (good for created_at). auto_now=True updates the field every time the instance is saved (good for updated_at). Both make the field non-editable in forms.

Q. How do you debug a memory leak in Django?

A. Look for unbounded caches, large in-memory queries (use .iterator() to stream), file handles not being closed, and global mutable state. Profile RSS over time with tracemalloc or memory_profiler; if it grows under steady load, capture heap snapshots with objgraph or guppy. Watch DEBUG=True — it logs every SQL query in memory.

Q. How do you implement role-based access control?

A. Define Groups (e.g. 'editors', 'admins') and assign permissions to them; assign users to groups. Check with user.groups, user.has_perm('app.codename'), or PermissionRequiredMixin. For more complex RBAC, use django-guardian (per-object permissions) or django-rules (predicate-based).

Q. How would you build a real-time chat feature?

A. Use Django Channels with WebSocket consumers, a Redis channel layer for fan-out, and Daphne/Uvicorn as the ASGI server. Persist messages in Postgres; broadcast new messages to all connected clients in a room group. Auth via the session middleware in the channels routing stack.

Q. How do you prevent race conditions on a counter / inventory field?

A. Use F() expressions: Item.objects.filter(pk=id).update(stock=F('stock')-1) — the DB does the math atomically. For more complex flows, use select_for_update() inside transaction.atomic() to lock the row until commit.

Q. How do you scale a Django app horizontally?

A. Make it stateless: move session and cache to Redis, files to S3, DB to managed Postgres. Run many Gunicorn instances behind a load balancer. Push background work to Celery workers (also horizontal). Use a read replica for heavy reads. Cache aggressively. Add a CDN in front of static assets.

Q. What is a common reason a migration is irreversible?

A. Dropping data — e.g. removing a column with values, deleting model rows in a data migration without a corresponding reverse. Always provide a reverse function in RunPython, or accept the irreversibility consciously with migrations.RunPython.noop.

21. Coding Problems & Solutions

Below are typical hands-on tasks asked in coding rounds for junior & associate full-stack roles using Django.

Problem 1: Build a Blog API

Requirements: CRUD posts; only authors can edit their own; JWT auth; paginated list; filter by status; search by title.

```
# models.py
class Post(models.Model):
    title = models.CharField(max_length=200)
    body = models.TextField()
    author = models.ForeignKey(User, on_delete=models.CASCADE,
                               related_name="posts")
    status = models.CharField(max_length=10,
                              choices=[("draft", "Draft"),
                                       ("published", "Published")],
                              default="draft")
    created_at = models.DateTimeField(auto_now_add=True)
    class Meta: ordering = ["-created_at"]

# serializers.py
class PostSerializer(serializers.ModelSerializer):
    author = serializers.ReadOnlyField(source="author.username")
    class Meta:
        model = Post
        fields = ["id", "title", "body", "author", "status", "created_at"]

# permissions.py
class IsOwnerOrReadOnly(permissions.BasePermission):
    def has_object_permission(self, request, view, obj):
        return (request.method in permissions.SAFE_METHODS
                or obj.author == request.user)

# views.py
class PostViewSet(viewsets.ModelViewSet):
    queryset = Post.objects.select_related("author")
    serializer_class = PostSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly,
                          IsOwnerOrReadOnly]
    filterset_fields = ["status"]
    search_fields = ["title"]
    def perform_create(self, serializer):
        serializer.save(author=self.request.user)
```

Problem 2: Email Verification on Signup

```
# After user is created, send a signed token via Celery
from django.core.signing import TimestampSigner
from django.urls import reverse
from .tasks import send_verification_email

@receiver(post_save, sender=User)
def signup_email(sender, instance, created, **kwargs):
    if created and not instance.is_active:
        token = TimestampSigner().sign(instance.pk)
        send_verification_email.delay(instance.email, token)

# view
def verify_email(request, token):
```

```

try:
    pk = TimestampSigner().unsign(token, max_age=60*60*24)
except (BadSignature, SignatureExpired):
    return HttpResponseBadRequest("Invalid or expired link")
User.objects.filter(pk=pk).update(is_active=True)
return redirect("login")

```

Problem 3: Top-N Authors by Post Count

```

from django.db.models import Count

top = (User.objects
        .annotate(post_count=Count("posts"))
        .filter(post_count__gt=0)
        .order_by("-post_count")[:10])

for u in top:
    print(u.username, u.post_count)

```

Problem 4: Rate-Limit Login Endpoint

```

# Using django-ratelimit
from django_ratelimit.decorators import ratelimit

@ratelimit(key="ip", rate="5/m", block=True)
def login_view(request):
    ...

# Or with DRF throttling
class LoginThrottle(AnonRateThrottle):
    rate = "5/min"

```

Problem 5: Bulk CSV Import without N+1

```

import csv
from io import TextIOWrapper

def import_csv(file):
    reader = csv.DictReader(TextIOWrapper(file, encoding="utf-8"))
    objs = [
        Post(title=row["title"], body=row["body"],
             author_id=int(row["author_id"]))
        for row in reader
    ]
    Post.objects.bulk_create(objs, batch_size=1000)
    return len(objs)

```

Problem 6: Soft Delete

```

class SoftDeleteQuerySet(models.QuerySet):
    def delete(self):
        return super().update(is_deleted=True, deleted_at=timezone.now())
    def alive(self):
        return self.filter(is_deleted=False)

class SoftDeleteManager(models.Manager):
    def get_queryset(self):
        return SoftDeleteQuerySet(self.model, using=self._db).alive()

class Post(models.Model):
    is_deleted = models.BooleanField(default=False)
    deleted_at = models.DateTimeField(null=True, blank=True)
    objects = SoftDeleteManager()

```

```
all_objects = models.Manager()
def delete(self, using=None, keep_parents=False):
    self.is_deleted = True
    self.deleted_at = timezone.now()
    self.save()
```

Problem 7: Custom Pagination Header

```
class CountPagination(PageNumberPagination):
    page_size = 20
    def get_paginated_response(self, data):
        return Response({
            "count": self.page.paginator.count,
            "next": self.get_next_link(),
            "prev": self.get_previous_link(),
            "results": data,
        })
```

22. System Design Basics

At the Associate level, you'll be asked to sketch simple systems. Practice walking the interviewer through the key components, the data model, scaling concerns, and trade-offs.

22.1 Standard System Design Checklist

- **Clarify requirements:** functional & non-functional. Read & write volume? Latency?
- **Data model:** tables, relations, indexes.
- **API design:** REST endpoints, request/response shapes.
- **Scaling:** caching, read replicas, sharding, queues.
- **Reliability:** monitoring, logging, retries, idempotency.
- **Security:** auth, authorization, rate limiting, encryption.

22.2 Example: URL Shortener

Components:

- Django + DRF for API
- Postgres for url storage
- Redis for hot-link cache
- Nginx for static and rate-limit

Data Model:

```
Url(id BIGSERIAL pk, code VARCHAR(8) UNIQUE, original TEXT,
    created_by FK->User NULL, created_at TIMESTAMP,
    hits BIGINT default 0)
```

API:

```
POST /api/shorten {url} -> {code, short_url}
GET /<code> -> 302 redirect
GET /api/stats/<code> -> {hits, created_at, original}
```

Algorithm:

- Insert row, generate code = base62(id)
- Cache code->original in Redis (TTL 24h)
- On redirect: check cache first, fall back to DB, F('hits')+1

Scaling:

- Read-heavy => add read replica + Redis
- Use Celery to async-update hit counts in batches
- Add Cloudflare in front for global CDN

22.3 Example: Blog with Real-Time Comments

Stack:

- Django + DRF for REST
- Channels + Daphne for WebSocket /ws/post/<id>/
- Redis channel layer for fan-out
- Postgres for persistence
- Celery for notifications & emails

Flow:

- Client posts comment via REST (persisted in DB)
- Signal/post_save broadcasts to channel group "post_<id>"
- All subscribed clients receive comment over WS
- Celery sends email notifications to other commenters

Indexes:

- `Comment(post_id, created_at)` composite index for feeds

23. Mock Interview Scenarios

Scenario 1: "Walk me through your last Django project."

Structure your answer:

- **Context** (1 sentence): what was the project & who was it for?
- **Stack**: Django version, DB, frontend, deployment.
- **Your role**: what you specifically built.
- **One tough problem** you solved (be specific).
- **Trade-offs** you made & what you'd do differently.
- **Impact**: users, performance, business metric.

Scenario 2: "Code Review" Question

Interviewer shows code; you spot bugs. Common issues to look for:

- N+1 queries inside a for loop (missing `select_related`).
- Using `filter().first()` where `get_object_or_404` would be clearer.
- Missing CSRF token on a form, or `@csrf_exempt` without auth.
- Building SQL with f-strings (SQL injection).
- Catching bare except: without logging.
- No pagination on a list endpoint.
- Hardcoded `SECRET_KEY` or DB password.
- Race condition: read-modify-write instead of `F()` expression.

Scenario 3: Live Debug

"This endpoint returns 500 in production but works locally."

- Check Sentry / logs first — read the actual traceback.
- `DEBUG=False` is hiding the error; reproduce with a local `DEBUG=True` against staging.
- Common culprits: missing migration, missing env var, `ALLOWED_HOSTS`, missing static collection, permissions on `/media`.
- Reproduce in a Django shell against the same DB if possible.

Scenario 4: Behavioural

Use the **STAR** framework: Situation → Task → Action → Result. Have 4–5 stories prepared:

- Time you fixed a production bug.
- Time you disagreed with a teammate.
- Time you took initiative.
- Time you missed a deadline & what you learned.
- Time you mentored someone or improved a process.

24. Final Checklist & Resources

24.1 Skills Self-Assessment

Tick each item before the interview. If unsure, revisit the chapter.

- I can scaffold a project, app, and run migrations from scratch.
- I can write models with FK, O2O, M2M and explain `on_delete`.
- I can write efficient ORM queries using `select_related` / `prefetch_related`.
- I can use Q, F, aggregations and annotations.
- I can write FBVs and CBVs (ListView, DetailView, CreateView, etc.).
- I can build a complete REST API with DRF: serializers, viewsets, routers.
- I can implement JWT or token auth with permissions and throttling.
- I understand sessions, cookies, CSRF and Django's security defaults.
- I can write a custom user model and explain why it must be done early.
- I can use Django signals appropriately (and know when not to).
- I can write middleware and a custom management command.
- I can cache with Redis at low level, per-view, and template-fragment.
- I can offload long jobs to Celery and schedule with Celery Beat.
- I can write model tests, view tests, and DRF API tests.
- I can dockerise a Django app and write a docker-compose file.
- I can deploy with Gunicorn + Nginx, configure HTTPS, and run `collectstatic`.
- I can read EXPLAIN ANALYZE output and add proper indexes.
- I can integrate a React or Vue frontend via JSON APIs & CORS.
- I can answer all 80+ Q&A in this guide without re-reading.
- I have built & deployed at least one end-to-end project I can demo.

24.2 Recommended Practice Projects

- **Personal Blog** with comments & auth (basics).
- **Task Manager** with REST API + React frontend (full stack).
- **E-commerce mini-app** with cart, checkout, Stripe.
- **Real-time chat** with Channels + Redis.
- **URL shortener** with Redis caching & analytics.
- **Job board** with search, filters, pagination, file uploads.

24.3 Resources

- **Official docs** — docs.djangoproject.com (read the topical guides).
- **DRF docs** — www.django-rest-framework.org.

- **Django Packages** — djangopackages.org for vetted libraries.
- **Two Scoops of Django** — practical patterns & anti-patterns.
- **HackerRank / LeetCode** — coding practice (Python + SQL).
- **System Design Primer** — github.com/donnemartin/system-design-primer.
- **Build & deploy** — actually ship a project; nothing beats it.

You're ready when...

you can take this entire guide and explain any chapter to a peer in your own words, your hands type the code without looking it up, and you've shipped at least one project end-to-end. Walk into the interview confident, be honest about what you don't know, and lean on real examples from your own code.

Good luck — you've got this!